CARAT: Class-Aware Robust Aggregation via Hidden Task Certificates for Poisoning-Resilient Federated Learning

Anonymous Author(s)
Affiliation
Address
email

Abstract

Federated learning servers must aggregate client updates without observing client data, which leaves training vulnerable to both untargeted model poisoning and targeted backdoor attacks. Existing robust aggregators mainly rely on update geometry, coordinate-wise robust statistics, or alignment with a single trusted reference direction, but these signals can be brittle when benign clients are heterogeneous and adaptive attackers stay near the benign update manifold. We present **CARAT**, a class-aware robust aggregation rule based on *hidden task certificates*: using server-side labeled reference data, CARAT instantiates multiple hidden, class-balanced micro-tasks and scores each client update by the loss reduction it induces across them. CARAT combines this task-aware signal with robust clipping, common-radius evaluation, coordinate-rank anomaly penalties, and capped-simplex weight optimization under a worst-task objective with temporal regularization, thereby favoring updates that remain broadly useful across hidden semantic slices while suppressing structurally extreme clients. In the strongest completed repository-backed CIFAR100/FangAttack pilot, CARAT suppresses attacker weights and substantially improves over Mean; however, because the current completed evidence is still training-only and the full clean-accuracy and ASR benchmark suite is not yet finished, these results should be read as initial empirical validation rather than a complete benchmark claim.

1 Introduction

Federated learning (FL) enables many clients to train a shared model without centralizing their raw data [1–3]. This decentralization is attractive in privacy- and governance-sensitive settings, but it also leaves the server in a fundamentally asymmetric position: it must update the global model using client messages that it cannot directly verify. Poisoning attacks exploit exactly this asymmetry. Malicious clients can slow convergence, distort the learned decision boundary, or implant triggers that activate attacker-chosen behavior at test time [4–11]. In practice, the robustness of FL is therefore tightly coupled to the aggregation rule.

The central difficulty is that malicious behavior and benign heterogeneity can look superficially similar. Under non-i.i.d. partitions, benign clients legitimately differ in direction, norm, sparsity, and class emphasis [12–15]. As a result, an update can appear unusual either because it is harmful or because it reflects a different local data distribution. Existing defenses often rely on signals that are too coarse to resolve this ambiguity reliably. Pairwise distances and geometric centrality can misclassify benign dispersion as adversarial deviation; coordinate-wise robust statistics suppress only marginal extremes and ignore cross-coordinate structure; and trusted-reference schemes reduce the

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

server's auxiliary information to a single global direction, which can miss failures concentrated on particular classes or semantic regions [16–23].

This leaves a key question: how should the server validate client updates when benign heterogeneity is intrinsic and the most harmful attacks may only reveal themselves on specific semantic slices of the task? What is needed is a validation signal that is semantically discriminative, robust to benign diversity, and difficult for attackers to satisfy by merely staying near the benign update manifold.

To address this, the paper proposes **CARAT**, *Class-Aware Robust Aggregation via Hidden Task Certificates*. CARAT assumes server-side access to labeled reference data and uses that data to instantiate hidden, class-balanced micro-tasks. Each client update is then scored by the loss reduction it induces across these tasks, producing a certificate matrix that measures whether the update is broadly useful across hidden semantic slices rather than merely geometrically central. CARAT combines this task-aware signal with robust clipping, common-radius evaluation, coordinate-rank anomaly penalties, and capped-simplex weight optimization under a worst-task objective with temporal regularization. Our main contributions are summarized as follows.

- We propose CARAT, a class-aware robust aggregation rule that validates client updates through hidden-task certificates built from server-side labeled reference data, replacing one-shot geometric trust signals with multi-task utility checks.
- We formulate aggregation as a capped-simplex optimization problem that combines worst-task utility across hidden tasks, coordinate-rank anomaly suppression, and temporal regularization, yielding a robust weighting mechanism that remains compatible with heterogeneous FL.
- We provide repository-backed empirical evidence for CARAT and clearly distinguish completed verified runs from the broader clean-accuracy and ASR benchmark suite that is still being finalized.

2 Related Work

Model poisoning attacks. Modern FL attacks are explicitly designed to remain effective under server-side robustness measures. Early work already showed that highly constrained adversaries can poison the global model while preserving substantial stealth, even when the server deploys Byzantine-style defenses [4–6]. On the untargeted side, MinMax, MinSum, ALIE, and related optimization-based attacks craft malicious updates that evade simple geometric or statistical filters while still degrading global learning [6, 7]. On the targeted side, model-replacement backdoors, distributed backdoor construction, trigger-tail attacks, and durable attacks such as Neurotoxin show that an update can appear globally plausible while implanting highly specific malicious behavior [8–11]. Recent surveys make the broader point explicit: once the server cannot inspect raw client data, poisoning and backdoor attacks become a first-class systems concern rather than an edge case [3, 24].

Defense and detection. Classical Byzantine-robust aggregation rules infer benignness from centrality. Krum and related geometry-based methods favor updates that remain close to other clients under pairwise distances [16, 17], while coordinate-wise median and trimmed mean suppress marginal extremes independently [18]. Later variants extend this family through suspicion scoring, regularized robust aggregation, coding-based redundancy, and geometric-median aggregation [19–22, 25]. These methods are appealing because they require limited server-side prior knowledge, but under heterogeneous data they may still confuse legitimate benign dispersion with malicious deviation. This limitation has motivated a second line of work that explicitly tackles heterogeneity, for example through FedProx- and SCAFFOLD-style stabilization, resampling, bucketing, local-SGD analysis, gradient splitting, nearest-neighbor mixing, label-skew-aware aggregation, and entropy-based public-data guidance [13–15, 26–32]. A third line augments aggregation with trusted data, history, or representation-level inspection. In collaborative-learning settings, AUROR already anticipated update screening for poisoning resistance [33]; in FL, FLTrust introduces a trusted server anchor, FoolsGold and FLDetector rely on historical similarity diagnostics, and FLARE, FLAME, DeepSight, CRFL, and FLIP use latent-space inspection, clustering, clipping, smoothing, or reverse-engineering-based backdoor mitigation [23, 34–40]. Recent work also revisits the cost of robustness itself, highlighting clipping design and the tradeoff between Byzantine resilience and no-attack accuracy [41, 42].

CARAT is closest to this broader defense-and-detection family, but differs in its validation principle: instead of trusting update similarity, one reference direction, or post-hoc clustering alone, it evaluates whether a client update remains useful across multiple hidden semantic sub-tasks.

3 Method

3.1 Formulation and Assumptions

We consider synchronous FL with n clients and a global parameter vector $w_r \in \mathbb{R}^d$ at round r. At each round, client i returns a message u_i^r, which the server maps to a vector-form update $\Delta_i^r \in \mathbb{R}^d$. For FedAvg, Δ_i^r can be read as the client model delta relative to the current global model; for gradient-style optimizers, it is the client update itself. The server must aggregate $\{\Delta_i^r\}_{i=1}^n$ into a single global step while a fraction ρ of clients may be malicious and may coordinate their behavior.

CARAT assumes the server has access to labeled reference data $\mathcal{D}^{\mathrm{ref}}$. From this data, it constructs a probe pool $\mathcal{P}$ and repeatedly samples hidden, class-balanced validation tasks. This is a stronger assumption than fully data-free robust aggregation, and it should be treated as part of the method definition rather than as an incidental implementation detail. The reason for making this assumption explicit is that CARAT uses the server's auxiliary data not to define one trusted direction, but to generate many hidden semantic checks against which candidate client updates can be evaluated.

The goal is to assign aggregation weights that preserve benign diversity while suppressing updates that are either semantically harmful or structurally anomalous. This distinction matters in non-i.i.d. FL: benign clients may differ substantially in direction, norm, and class emphasis, so geometric dispersion alone is not enough to identify attacks. CARAT therefore separates each round into three conceptual steps. It first places all client updates into a common evaluation geometry, then measures each client's task utility and structural abnormality, and finally converts these signals into robust aggregation weights. This separation is central to the method: evaluation should be comparable across clients, while the final server step should still remain conservative and robust.

3.2 Method Details

At round r, CARAT begins by converting all client messages into vector-form deltas and computing their norms. Before any semantic evaluation, the server robustly clips client updates to prevent a small number of large-magnitude messages from dominating the comparison stage. Let $\|\Delta_i\|_2$ denote the norm of client i's update. In our implementation, the clipping threshold τ_r is estimated by a MAD-based rule:

$$\tau_r = \mathrm{median}(\{\|\Delta_i\|_2\}) + k \cdot 1.4826 \cdot \mathrm{MAD}(\{\|\Delta_i\|_2\}), \tag{1}$$

and the clipped update is

$$\bar{\Delta}_i = \min\left(1, \frac{\tau_r}{\|\Delta_i\|_2 + \varepsilon}\right) \Delta_i. \tag{2}$$

CARAT then rescales each clipped update to a common evaluation radius q_r:

$$\tilde{\Delta}_i = q_r \frac{\bar{\Delta}_i}{\|\bar{\Delta}_i\|_2 + \varepsilon}, \tag{3}$$

where q_r is chosen as a quantile of the positive clipped norms. This normalization removes pure scale advantages during hidden-task comparison. Importantly, the normalized vectors are used only for evaluation; the final model update is still computed from the clipped vectors. In other words, CARAT uses normalization to create a fair scoring geometry, not to define the actual optimization step taken by the server.

After this preprocessing stage, CARAT samples T hidden tasks from the probe pool $\mathcal{P}$, with each task constructed to be approximately class-balanced. This is the source of the method's class-aware behavior. A malicious update can remain globally plausible while degrading only a subset of classes or semantic regions, whereas repeated balanced hidden tasks make these localized failures harder to hide. For each hidden task t, let $\ell_t(w)$ denote the average cross-entropy loss of model w on that task. CARAT defines a *hidden-task certificate*

$$c_{i,t} = \ell_t(w_r) - \ell_t(w_r + \tilde{\Delta}_i). \tag{4}$$

Here, "certificate" denotes an internal task-level utility score rather than a formal certified-robustness guarantee. Positive values indicate that client i improves task t relative to the current global model.

Stacking these scores yields a certificate matrix $C \in \mathbb{R}^{n \times T}$. Because different tasks may have different intrinsic difficulty and baseline loss scale, CARAT robustly standardizes each column of C before optimization. This preserves within-task ordering while making the task scores comparable across different hidden slices. The resulting matrix gives the server a much richer signal than client-to-client similarity alone: a client should be trusted because it remains broadly useful across hidden tasks, not merely because it lies near the geometric center of the client population.

Task utility alone is still insufficient, because a malicious client may overfit to the probe tasks or align with part of the semantic signal while remaining structurally extreme in parameter space. CARAT therefore augments hidden-task certificates with a coordinate-rank anomaly score. For a sampled coordinate set $S \subseteq \{1, \ldots, d\}$, it ranks clients coordinate-wise and measures the average normalized distance from the center rank:

$$r_i^{(S)} = \frac{1}{|S|} \sum_{j \in S} \left| \frac{\mathrm{rank}_{i,j}}{n-1} - \frac{1}{2} \right|. \tag{5}$$

CARAT repeats this computation over several random coordinate subsets and aggregates the resulting values robustly. A client receives a large anomaly score if its update remains persistently extreme across many coordinates rather than only on a few isolated entries. After robust standardization, only positive deviations are retained as penalties. The signal is intentionally asymmetric: suspicious extremeness is penalized, whereas unusual centrality is not explicitly rewarded. Together, the certificate matrix and rank penalty separate two notions that are often conflated in robust aggregation: whether an update is useful, and whether it remains structurally atypical.

3.3 Weight Optimization and Aggregation

CARAT converts these signals into aggregation weights by solving a constrained optimization problem on the capped simplex

$$\mathcal{A}(\hat{\rho}) = \left\{ \alpha \in \mathbb{R}^n : \alpha_i \geq 0,\ \sum_{i=1}^{n} \alpha_i = 1,\ \alpha_i \leq \frac{1}{(1-\hat{\rho})n} \right\}, \tag{6}$$

where $\hat{\rho}$ is an estimate of the attacker fraction. The cap prevents any single client from dominating the aggregate and encodes a conservative trust budget at the weight level: even when a client appears useful, it cannot absorb arbitrary influence in one round.

Let $r \in \mathbb{R}^n$ denote the positive rank penalties and let α^{prev} be the previous round's weights. CARAT solves

$$\max_{\alpha \in \mathcal{A}(\hat{\rho})} \ \operatorname*{softmin}_{\beta}(C^\top \alpha) - \lambda_r r^\top \alpha - \lambda_p \|\alpha - \alpha^{\mathrm{prev}}\|_2^2, \tag{7}$$

where

$$\operatorname*{softmin}_{\beta}(s_1, \ldots, s_T) = -\frac{1}{\beta} \log \left(\frac{1}{T} \sum_{t=1}^{T} e^{-\beta s_t} \right). \tag{8}$$

The first term is a smooth approximation to worst-task utility: large β places more emphasis on the weakest hidden tasks, while finite β preserves optimization stability. The second term penalizes structurally extreme clients, and the third term regularizes the solution toward the previous round to reduce unnecessary oscillation. Eq. (7) therefore encodes a clear decision rule: a client should retain influence only if it remains useful across hidden tasks and does not repeatedly appear as an outlier in rank space.

After optimization, CARAT aggregates the clipped updates:

$$\Delta_{\mathrm{agg}} = \sum_{i=1}^{n} \alpha_i \bar{\Delta}_i. \tag{9}$$

This yields the final server step. The final update therefore inherits the robustness benefits of clipping while using weights learned from the richer evaluation geometry defined above. In practice, the main additional computation comes from evaluating n candidate updates over T hidden tasks and

computing rank penalties on coordinate subsamples; the probe-model forward passes dominate, while the ranking and optimization stages remain tunable through explicit budget parameters.

A compact round-level algorithm summary is provided in Appendix A. Formal properties of the normalization step, capped-simplex feasibility, softmin behavior, temporal regularization, and per-round complexity are deferred to Appendix B.

4 Experiments

4.1 Experimental Setup

Datasets, models, and heterogeneity. We evaluate CARAT on CIFAR-10 with VGG19 [43] and on CIFAR-100 with ResNet18 [44]. To study robustness under client heterogeneity, we partition client data with a Dirichlet distribution $\boldsymbol{p} \sim \mathrm{Dir}_N(\alpha)$ and report non-IID settings with $\alpha \in \{0.1, 0.5, 1\}$. The main-text tables and figures focus on the completed untargeted-poisoning slice of this benchmark; broader benchmark scope and implementation details are deferred to Appendix C.

Baselines, attacks, and default training setting. Unless otherwise stated, we compare CARAT against FLTrust [23], FLDetector [35], NormClipping, and MultiKrum [16], and we include a no-attack Mean reference where useful. The current main-text robustness results use four untargeted model-poisoning attacks: ALIE [7], Min-Max [6], Min-Sum [6], and FangAttack [5]. The shared training configuration is summarized in Table 1; CARAT-specific defaults and the full repository benchmark matrix are recorded in Appendix C.

Table 1: Default training configuration for the main-text experiments.

Parameter	**Total Clients** (N)	**Malicious Clients Ratio** ($\%$)	**Clients per Round**	**Local Iterations** (K)	**Total Epochs** (T)	**Learning Rate** (η)	**Significance Level**
Value	20	10/20/30	20	4	200	0.05	0.02

4.2 Untargeted Poisoning Results

Table 2 reports the completed untargeted-poisoning results with 10% malicious clients. Across the reported non-IID settings, `CARAT` attains the best average rank and remains close to the no-attack Mean reference. The clearest gap appears on CIFAR-100/ResNet18 under FangAttack with $\alpha = 0.1$, where `CARAT` reaches 0.0323 test loss versus 0.0504 for MultiKrum and 0.0450 for FLTrust.

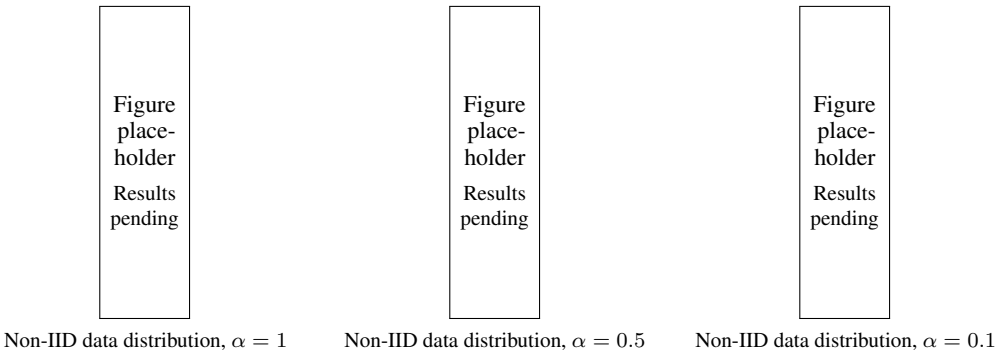


Figure 1: Test-accuracy trajectories on CIFAR-100/ResNet18 under untargeted poisoning with 20 clients and 4 attackers.

Fig. 1 shows the same pattern on CIFAR-100/ResNet18: `CARAT` tracks the no-attack reference more closely than the competing defenses, while FLTrust and MultiKrum show larger degradation and instability. The gap widens as heterogeneity increases.

Impact of Attacker Ratio. We next increase the attacker ratio from the default 10% to 20% and 30%, keeping the remaining settings fixed. Fig. 2 shows that all methods degrade as the adversarial

Table 2: Final test loss under four untargeted model-poisoning attacks with 10% malicious clients on two dataset-model pairs under non-IID settings.

Dataset Model	Non-IID Settings	Defense Rules	Attacks				Rank
			ALIE	FangAttack	MinMax	MinSum	
CIFAR-10 VGG19	$\alpha = 1$	NormClipping	0.0176±0.0003	0.0184±0.0003	0.0176±0.0003	0.0177±0.0003	5.0
		MultiKrum	0.0120±0.0001	0.0159±0.0009	0.0157±0.0005	0.0153±0.0010	4.0
		FLTrust	0.0152±0.0003	0.0152±0.0003	0.0148±0.0003	0.0149±0.0003	3.0
		FLDetector	**0.0093±0.0002**	0.0192±0.0018	**0.0089±0.0001**	**0.0089±0.0002**	2.0
		CARAT	**0.0093±0.0002**	**0.0093±0.0002**	0.0090±0.0002	0.0090±0.0002	1.0
		NoAttack+Mean(Base)	0.0092±0.0001	0.0092±0.0001	0.0092±0.0001	0.0092±0.0001	–
	$\alpha = 0.5$	NormClipping	0.0194±0.0002	0.0200±0.0004	0.0196±0.0004	0.0195±0.0003	4.5
		MultiKrum	0.0147±0.0008	0.0207±0.0021	0.0196±0.0012	0.0208±0.0023	4.5
		FLTrust	0.0167±0.0004	0.0168±0.0003	0.0167±0.0004	0.0163±0.0003	3.0
		FLDetector	0.0108±0.0005	0.0242±0.0005	0.0110±0.0010	0.0106±0.0006	2.0
		CARAT	**0.0106±0.0003**	**0.0107±0.0003**	**0.0107±0.0010**	**0.0104±0.0003**	1.0
		NoAttack+Mean(Base)	0.0104±0.0004	0.0104±0.0004	0.0104±0.0004	0.0104±0.0004	–
	$\alpha = 0.1$	NormClipping	0.0256±0.0011	0.0261±0.0012	0.0268±0.0013	0.0260±0.0015	4.0
		MultiKrum	0.0270±0.0013	0.0330±0.0028	0.0321±0.0034	0.0327±0.0026	5.0
		FLTrust	0.0233±0.0011	0.0236±0.0013	0.0243±0.0018	0.0239±0.0013	3.0
		FLDetector	0.0215±0.0022	0.0243±0.0022	0.0218±0.0031	0.0218±0.0034	2.0
		CARAT	**0.0198±0.0016**	**0.0198±0.0018**	**0.0198±0.0016**	**0.0197±0.0024**	1.0
		NoAttack+Mean(Base)	0.0199±0.0012	0.0199±0.0012	0.0199±0.0012	0.0199±0.0012	–
CIFAR-100 ResNet18	$\alpha = 1$	NormClipping	0.0473±0.0003	0.0487±0.0004	0.0479±0.0003	0.0480±0.0003	5.0
		MultiKrum	0.0330±0.0004	0.0374±0.0008	0.0375±0.0005	0.0372±0.0007	3.0
		FLTrust	0.0405±0.0003	0.0405±0.0004	0.0403±0.0005	0.0404±0.0004	4.0
		FLDetector	**0.0274±0.0005**	0.0426±0.0004	0.0275±0.0004	**0.0275±0.0005**	2.0
		CARAT	0.0276±0.0004	**0.0276±0.0004**	**0.0273±0.0003**	**0.0275±0.0002**	1.0
		NoAttack+Mean(Base)	0.0269±0.0003	0.0269±0.0003	0.0269±0.0003	0.0269±0.0003	–
	$\alpha = 0.5$	NormClipping	0.0482±0.0006	0.0496±0.0003	0.0488±0.0004	0.0490±0.0005	5.0
		MultiKrum	0.0352±0.0008	0.0403±0.0015	0.0405±0.0012	0.0402±0.0010	3.0
		FLTrust	0.0414±0.0005	0.0414±0.0004	0.0412±0.0004	0.0413±0.0005	4.0
		FLDetector	**0.0284±0.0004**	0.0437±0.0011	**0.0284±0.0004**	**0.0283±0.0004**	2.0
		CARAT	**0.0284±0.0004**	**0.0285±0.0005**	**0.0284±0.0006**	**0.0283±0.0005**	1.0
		NoAttack+Mean(Base)	0.0276±0.0005	0.0276±0.0005	0.0276±0.0005	0.0276±0.0005	–
	$\alpha = 0.1$	NormClipping	0.0519±0.0004	0.0529±0.0004	0.0527±0.0005	0.0529±0.0004	5.0
		MultiKrum	0.0424±0.0010	0.0504±0.0010	0.0504±0.0015	0.0504±0.0014	4.0
		FLTrust	0.0450±0.0004	0.0450±0.0005	0.0450±0.0005	0.0451±0.0006	3.0
		FLDetector	0.0326±0.0003	0.0461±0.0039	0.0325±0.0003	**0.0323±0.0005**	2.0
		CARAT	**0.0322±0.0004**	**0.0323±0.0004**	**0.0322±0.0005**	**0.0323±0.0005**	1.0
		NoAttack+Mean(Base)	0.0314±0.0004	0.0314±0.0004	0.0314±0.0004	0.0314±0.0004	–

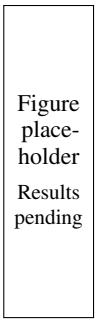
fraction grows, but the degradation for `CARAT` is consistently milder across the reported α values, whereas FLTrust and MultiKrum become unstable in part of this regime.

Scaling with More Clients. To test whether the method remains effective beyond the default 20-client setting, we increase the total number of clients to 40 and 60 while maintaining an attacker ratio of 20%. Fig. 3 and Table 3 show that `CARAT` continues to converge reliably and retains the best average rank across the scaled CIFAR-100 settings. Competing defenses degrade more sharply, especially under stronger heterogeneity or FangAttack.

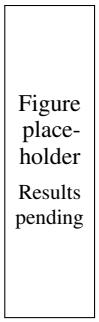
5 Conclusion

We presented CARAT, a class-aware robust aggregation rule that validates client updates through hidden task certificates rather than relying only on geometric centrality or a single trusted reference direction. By scoring updates across multiple hidden, class-balanced semantic slices of the task and combining those scores with rank-space anomaly suppression and capped-simplex weight optimization, CARAT is designed to preserve benign heterogeneity while reducing the influence of semantically harmful or structurally extreme clients.

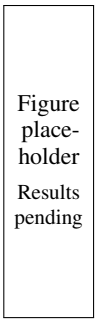
The current repository-backed evidence supports this mechanism claim at a preliminary empirical level: in the completed CIFAR100/FangAttack pilot, CARAT suppresses attacker weights and



Dataset distribution is non-IID with $\alpha = 1$.

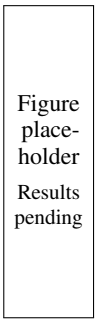


Dataset distribution is non-IID with $\alpha = 0.5$.

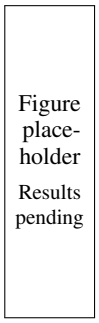


Dataset distribution is non-IID with $\alpha = 0.1$.

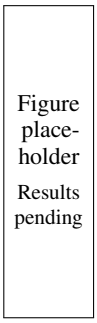
Figure 2: Sensitivity of test accuracy to the attacker ratio on CIFAR-100/ResNet18 under non-IID settings.



Dataset distribution is non-IID with $\alpha = 1$.



Dataset distribution is non-IID with $\alpha = 0.5$.



Dataset distribution is non-IID with $\alpha = 0.1$.

Figure 3: Sensitivity of test accuracy to the number of clients on CIFAR-100/ResNet18 under non-IID settings.

substantially improves over Mean. At the same time, the present manuscript does not yet support a full benchmark claim, because the clean-accuracy, ASR, non-i.i.d., and multi-seed CARAT suites are still being completed, and the method retains the explicit assumption of server-side labeled reference data. Even with these limitations, the hidden-task certificate view provides a concrete and technically grounded direction for robust FL aggregation, especially in settings where class-selective failures matter.

References

[1] H. Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Agüera y Arcas. Communication-efficient learning of deep networks from decentralized data. In *Artificial Intelligence and Statistics*, 2017.

[2] Kallista A. Bonawitz, Vladimir Ivanov, Ben Kreuter, Antonio Marcedone, H. Brendan McMahan, Sarvar Patel, Daniel Ramage, Aaron Segal, and Karn Seth. Practical secure aggregation for privacy-preserving machine learning. In *ACM SIGSAC Conference on Computer and Communications Security*, pages 1175–1191, 2017.

[3] Peter Kairouz, H. Brendan McMahan, Brendan Avent, Aurelien Bellet, Mehdi Bennis, Arjun Nitin Bhagoji, Kallista Bonawitz, Zachary Charles, Graham Cormode, Rachel Cummings, et al. *Advances and Open Problems in Federated Learning*. Now Publishers, 2021.

[4] Arjun Nitin Bhagoji, Supriyo Chakraborty, Prateek Mittal, and Seraphin Calo. Analyzing federated learning through an adversarial lens. In *International Conference on Machine Learning*, pages 634–643, 2019.

[5] Minghong Fang, Xiaoyu Cao, Jinyuan Jia, and Neil Zhenqiang Gong. Local model poisoning attacks to Byzantine-robust federated learning. In *USENIX Security Symposium*, 2020.

Table 3: Final test loss on CIFAR-100/ResNet18 under four untargeted model-poisoning attacks at adversary ratio 20%, comparing non-IID settings across different client counts. Each cell reports mean±std over available seeds. If a run stopped early, its last recorded epoch before 200 is used.

Number of Clients	Non-IID Settings	Defense Rules	Attacks				Rank
			ALIE	FangAttack	MinMax	MinSum	
40	$\alpha = 1$	NormClipping	0.0472±0.0003	0.0586±0.0067	0.0479±0.0002	0.0484±0.0003	5.0
		MultiKrum	0.0427±0.0012	0.0309±0.0006	0.0268±0.0007	0.0284±0.0013	3.0
		FLTrust	0.0412±0.0002	0.0405±0.0001	0.0403±0.0001	0.0405±0.0003	4.0
		FLDetector	0.0299±0.0007	0.0601±0.0024	0.0259±0.0005	0.0260±0.0006	2.0
		CARAT	**0.0276±0.0006**	**0.0258±0.0006**	**0.0257±0.0007**	**0.0257±0.0005**	1.0
		NoAttack+Mean(Base)	0.0251±0.0006	0.0251±0.0006	0.0251±0.0006	0.0251±0.0006	–
	$\alpha = 0.5$	NormClipping	0.0482±0.0005	0.0560±0.0005	0.0490±0.0006	0.0496±0.0008	5.0
		MultiKrum	0.0448±0.0008	0.0327±0.0005	0.0279±0.0005	0.0293±0.0013	3.0
		FLTrust	0.0422±0.0002	0.0415±0.0004	0.0414±0.0005	0.0416±0.0004	4.0
		FLDetector	0.0311±0.0006	0.0572±0.0032	0.0267±0.0003	0.0268±0.0004	2.0
		CARAT	**0.0290±0.0004**	**0.0267±0.0004**	**0.0266±0.0003**	**0.0265±0.0002**	1.0
		NoAttack+Mean(Base)	0.0259±0.0003	0.0259±0.0003	0.0259±0.0003	0.0259±0.0003	–
	$\alpha = 0.1$	NormClipping	0.0524±0.0004	0.0602±0.0005	0.0537±0.0006	0.0544±0.0006	5.0
		MultiKrum	0.0552±0.0007	0.0419±0.0010	0.0346±0.0005	0.0370±0.0014	3.0
		FLTrust	0.0466±0.0005	0.0458±0.0005	0.0460±0.0005	0.0468±0.0005	4.0
		FLDetector	0.0373±0.0009	0.0550±0.0043	0.0315±0.0008	0.0315±0.0006	2.0
		CARAT	**0.0345±0.0006**	**0.0305±0.0004**	**0.0311±0.0007**	**0.0308±0.0005**	1.0
		NoAttack+Mean(Base)	0.0303±0.0006	0.0303±0.0006	0.0303±0.0006	0.0303±0.0006	–
60	$\alpha = 1$	NormClipping	0.0470±0.0003	0.0593±0.0037	0.0484±0.0012	0.0486±0.0003	5.0
		MultiKrum	0.0452±0.0003	0.0303±0.0011	0.0258±0.0005	0.0275±0.0009	3.0
		FLTrust	0.0410±0.0003	0.0427±0.0027	0.0401±0.0003	0.0405±0.0005	4.0
		FLDetector	0.0290±0.0003	0.0484±0.0007	**0.0247±0.0003**	0.0249±0.0004	2.0
		CARAT	**0.0271±0.0003**	**0.0260±0.0011**	**0.0247±0.0003**	**0.0247±0.0002**	1.0
		NoAttack+Mean(Base)	0.0245±0.0004	0.0245±0.0004	0.0245±0.0004	0.0245±0.0004	–
	$\alpha = 0.5$	NormClipping	0.0479±0.0004	0.0575±0.0011	0.0487±0.0003	0.0496±0.0004	5.0
		MultiKrum	0.0472±0.0008	0.0316±0.0006	0.0273±0.0013	0.0289±0.0013	3.0
		FLTrust	0.0422±0.0005	0.0431±0.0021	0.0411±0.0005	0.0418±0.0005	4.0
		FLDetector	0.0303±0.0006	0.0449±0.0012	0.0260±0.0005	0.0260±0.0004	2.0
		CARAT	**0.0285±0.0006**	**0.0264±0.0004**	**0.0255±0.0003**	**0.0255±0.0003**	1.0
		NoAttack+Mean(Base)	0.0253±0.0004	0.0253±0.0004	0.0253±0.0004	0.0253±0.0004	–
	$\alpha = 0.1$	NormClipping	0.0518±0.0004	0.0641±0.0059	0.0530±0.0005	0.0538±0.0004	5.0
		MultiKrum	0.0563±0.0005	0.0389±0.0016	0.0327±0.0006	0.0355±0.0014	3.0
		FLTrust	0.0463±0.0007	0.0519±0.0090	0.0454±0.0006	0.0459±0.0007	4.0
		FLDetector	0.0361±0.0007	0.0552±0.0112	0.0304±0.0007	0.0304±0.0007	2.0
		CARAT	**0.0338±0.0011**	**0.0292±0.0015**	**0.0298±0.0005**	**0.0298±0.0006**	1.0
		NoAttack+Mean(Base)	0.0294±0.0006	0.0294±0.0006	0.0294±0.0006	0.0294±0.0006	–

[6] Virat Shejwalkar and Amir Houmansadr. Manipulating the Byzantine: Optimizing model poisoning attacks and defenses for federated learning. In *Network and Distributed System Security Symposium*, 2021.

[7] Gilad Baruch, Moran Baruch, and Yoav Goldberg. A little is enough: Circumventing defenses for distributed learning. In *Advances in Neural Information Processing Systems*, 2019.

[8] Eugene Bagdasaryan, Andreas Veit, Yiqing Hua, Deborah Estrin, and Vitaly Shmatikov. How to backdoor federated learning. In *International Conference on Artificial Intelligence and Statistics*, pages 2938–2948, 2020.

[9] Chulin Xie, Keli Huang, Pin-Yu Chen, and Bo Li. DBA: Distributed backdoor attacks against federated learning. In *International Conference on Learning Representations*, 2020.

[10] Hongyi Wang, Kartik Sreenivasan, Shashank Rajput, Harit Vishwakarma, Saurabh Agarwal, Jy-yong Sohn, Kangwook Lee, and Dimitris Papailiopoulos. Attack of the tails: Yes, you really can backdoor federated learning. In *Advances in Neural Information Processing Systems*, 2020.

[11] Zhengming Zhang, Ashwinee Panda, Linyue Song, Yaoqing Yang, Michael Mahoney, Prateek Mittal, Ramchandran Kannan, and Joseph Gonzalez. Neurotoxin: Durable backdoors in federated learning. In *International Conference on Machine Learning*, pages 26429–26446, 2022.

[12] Yue Zhao, Meng Li, Liangzhen Lai, Naveen Suda, et al. Federated learning with non-IID data. *arXiv preprint arXiv:1806.00582*, 2018.

[13] Tian Li, Anit Kumar Sahu, Manzil Zaheer, Maziar Sanjabi, Ameet Talwalkar, and Virginia Smith. Federated optimization in heterogeneous networks. In *Proceedings of Machine Learning and Systems*, 2020.

[14] Sai Praneeth Karimireddy, Satyen Kale, Mehryar Mohri, Sashank Reddi, Sebastian Stich, and Ananda Theertha Suresh. SCAFFOLD: Stochastic controlled averaging for federated learning. In *International Conference on Machine Learning*, pages 5132–5143, 2020.

[15] Hangyu Zhu, Jinjin Xu, Shiqing Liu, and Yaochu Jin. Federated learning on non-IID data: A survey. *Neurocomputing*, 465:371–390, 2021.

[16] Peva Blanchard, El Mahdi El Mhamdi, Rachid Guerraoui, and Julien Stainer. Machine learning with adversaries: Byzantine tolerant gradient descent. In *Advances in Neural Information Processing Systems*, 2017.

[17] El Mahdi El Mhamdi, Rachid Guerraoui, and Sébastien Rouault. The hidden vulnerability of distributed learning in Byzantium. In *International Conference on Machine Learning*, 2018.

[18] Dong Yin, Yudong Chen, Ramchandran Kannan, and Peter Bartlett. Byzantine-robust distributed learning: Towards optimal statistical rates. In *International Conference on Machine Learning*, 2018.

[19] Cong Xie, Sanmi Koyejo, and Indranil Gupta. Zeno: Distributed stochastic gradient descent with suspicion-based fault-tolerance. In *International Conference on Machine Learning*, pages 6893–6901, 2019.

[20] Lingjiao Chen, Hongyi Wang, Zachary Charles, and Dimitris Papailiopoulos. DRACO: Byzantine-resilient distributed training via redundant gradients. In *International Conference on Machine Learning*, pages 903–912, 2018.

[21] Liping Li, Wei Xu, Tianyi Chen, Georgios B. Giannakis, and Qing Ling. RSA: Byzantine-robust stochastic aggregation methods for distributed learning from heterogeneous datasets. In *AAAI Conference on Artificial Intelligence*, pages 1544–1551, 2019.

[22] Krishna Pillutla, Sham M. Kakade, and Zaid Harchaoui. Robust aggregation for federated learning. *IEEE Transactions on Signal Processing*, 70:1142–1154, 2022.

[23] Xiaoyu Cao, Minghong Fang, Jia Liu, Neil Zhenqiang Gong, and Yixin Gong. FLTrust: Byzantine-robust federated learning via trust bootstrapping. In *Network and Distributed System Security Symposium*, 2021.

[24] Pengrui Liu, Xiangrui Xu, and Wei Wang. Threats, attacks and defenses to federated learning: Issues, taxonomy and perspectives. *Cybersecurity*, 5(4), 2022.

[25] Dong Yin, Yudong Chen, Ramchandran Kannan, and Peter Bartlett. Defending against saddle point attack in byzantine-robust distributed learning. In *International Conference on Machine Learning*, pages 7074–7084, 2019.

[26] Lie He, Sai Praneeth Karimireddy, and Martin Jaggi. Byzantine-robust learning on heterogeneous datasets via resampling. In *International Conference on Learning Representations*, 2021.

[27] Sai Praneeth Karimireddy, Lie He, and Martin Jaggi. Byzantine-robust learning on heterogeneous datasets via bucketing. In *International Conference on Learning Representations*, 2022.

[28] Deepesh Data and Suhas Diggavi. Byzantine-resilient high-dimensional SGD with local iterations on heterogeneous data. In *International Conference on Machine Learning*, pages 2478–2488, 2021.

[29] Yuchen Liu, Chen Chen, Lingjuan Lyu, Fangzhao Wu, Sai Wu, and Gang Chen. Byzantine-robust learning on heterogeneous data via gradient splitting. In *International Conference on Machine Learning*, pages 21404–21425, 2023.

[30] Youssef Allouah, Sadegh Farhadkhani, Rachid Guerraoui, Nirupam Gupta, Rafael Pinot, and John Stephan. Fixing by mixing: A recipe for optimal byzantine ML under heterogeneity. In *International Conference on Artificial Intelligence and Statistics*, pages 1232–1300, 2023.

[31] Wenxuan Bao, Jun Wu, and Jingrui He. BOBA: Byzantine-robust federated learning with label skewness. In *International Conference on Artificial Intelligence and Statistics*, pages 892–900, 2024.

[32] Wenke Huang, Zekun Shi, Mang Ye, He Li, and Bo Du. Self-driven entropy aggregation for byzantine-robust heterogeneous federated learning. In *International Conference on Machine Learning*, pages 20096–20110, 2024.

[33] Shiqi Shen, Shruti Tople, and Prateek Saxena. AUROR: Defending against poisoning attacks in collaborative deep learning systems. In *Annual Computer Security Applications Conference*, 2016.

[34] Clement Fung, Chris J. M. Yoon, and Ivan Beschastnikh. The limitations of federated learning in sybil settings. In *International Symposium on Research in Attacks, Intrusions and Defenses*, 2020.

[35] Zaixi Zhang, Xiaoyu Xu, Jinyuan Jia, Matthew Jagielski, Gauri Joshi, Mingyi Hong, and Carl A. Gunter. FLDetector: Defending federated learning against model poisoning attacks via detecting malicious clients. In *ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2022.

[36] Ning Wang, Yang Xiao, Yimin Chen, Yang Hu, Wenjing Lou, and Y. Thomas Hou. FLARE: Defending federated learning against model poisoning attacks via latent space representations. In *ACM Asia Conference on Computer and Communications Security*, pages 946–958, 2022.

[37] Thien Duc Nguyen, Phillip Rieger, Hieu Yalame, Hauke Möllering, Huili Chen Xu, Ryohei Myung, Swarun Kadhe, and Kang G. Shin. FLAME: Taming backdoors in federated learning. In *USENIX Security Symposium*, 2022.

[38] Phillip Rieger, Thien Duc Nguyen, Hauke Möllering, Hieu Yalame, Huili Chen Xu, Ryohei Myung, Swarun Kadhe, and Kang G. Shin. DeepSight: Mitigating backdoor attacks in federated learning through deep model inspection. In *Network and Distributed System Security Symposium*, 2022.

[39] Chulin Xie, Minghao Chen, Pin-Yu Chen, and Bo Li. CRFL: Certifiably robust federated learning against backdoor attacks. In *International Conference on Machine Learning*, pages 11372–11382, 2021.

[40] Kaiyuan Zhang, Guanhong Tao, Qiuling Xu, Siyuan Cheng, Shengwei An, Yingqi Liu, Shiwei Feng, Guangyu Shen, Pin-Yu Chen, Shiqing Ma, and Xiangyu Zhang. FLIP: A provable defense framework for backdoor mitigation in federated learning. In *International Conference on Learning Representations*, 2023.

[41] Youssef Allouah, Rachid Guerraoui, Nirupam Gupta, Ahmed Jellouli, Geovani Rizk, and John Stephan. Adaptive gradient clipping for robust federated learning. In *International Conference on Learning Representations*, 2025.

[42] Yi-Rui Yang, Chang-Wei Shi, and Wu-Jun Li. On the tension between byzantine robustness and no-attack accuracy in distributed learning. In *International Conference on Machine Learning*, 2025.

[43] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. In *International Conference on Learning Representations*, 2015.

[44] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *IEEE Conference on Computer Vision and Pattern Recognition*, pages 770–778, 2016.

Algorithm 1 CARAT server aggregation at round r

Require: Current global model w_r, client messages $\{u_i^r\}_{i=1}^n$, previous-round weights α^{prev}, probe pool $\mathcal{P}$, CARAT hyperparameters
Ensure: Aggregated update Δ_{agg}, updated temporal prior α^{prev}

```
if P is uninitialized then
    Build a balanced probe pool from the server-visible training dataset
end if
if r = 1 then
    α^prev ← (1/n) 1
end if
for i = 1 to n do
    Convert u_i^r into a vector-form delta Δ_i^r
end for
Compute the clipping threshold τ_r from {‖Δ_i^r‖_2}_{i=1}^n
for i = 1 to n do
    Δ̄_i^r ← min(1, τ_r / (‖Δ_i^r‖_2 + ε)) Δ_i^r
end for
Choose the evaluation radius q_r from a quantile of positive clipped norms
for i = 1 to n do
    Δ̃_i^r ← q_r Δ̄_i^r / (‖Δ̄_i^r‖_2 + ε)
end for
Sample T class-balanced hidden tasks from P with replacement/subset fallback if strict balanced
sampling fails
for t = 1 to T do
    Evaluate the baseline loss ℓ_t(w_r)
end for
for i = 1 to n do
    for t = 1 to T do
        c_{i,t} ← ℓ_t(w_r) − ℓ_t(w_r + Δ̃_i^r)
    end for
end for
Robustly standardize certificate columns to obtain C^std
Compute rank penalties r from repeated coordinate subsamples of {Δ̄_i^r}_{i=1}^n
α^(0) ← α^prev
for k = 0 to K − 1 do
    z ← (C^std)^⊤ α^(k)
    π ← softmin task weights induced by z and β
    g ← C^std π − λ_r r − 2λ_p (α^(k) − α^prev)
    α^(k+1) ← Π_{A(ρ̂)}(α^(k) + ηg)
end for
α* ← α^(K) and α^prev ← α*
Δ_agg ← Σ_{i=1}^n α_i* Δ̄_i^r
return Δ_agg, α^prev
```

A Algorithmic Details

A.1 Round-Level Pseudocode

Algorithm 1 gives a standard round-level pseudocode view of one CARAT server round.

A.2 Implementation Notes

Inputs. Current global model w_r, client messages $\{u_i^r\}_{i=1}^n$, previous-round weights α^{prev}, server-visible training dataset, and CARAT hyperparameters.

Persistent state. CARAT maintains two pieces of state across rounds: a balanced probe pool sampled at initialization from the server-visible training dataset, and the previous-round weight vector α^{prev}, which serves as the temporal prior in the current round.

Initialization. At the first round, the implementation initializes α^{prev} to the uniform vector. This makes the initial round unbiased across clients while still allowing later rounds to exploit temporal consistency once CARAT has observed several optimization trajectories.

Reference-pool construction and hidden-task sampling. Let $\mathcal{P}_c \subseteq \mathcal{P}$ denote the subset of probe examples with class label c. A hidden task is constructed by sampling a small number of examples from each available class pool and concatenating them into one balanced micro-task. If the desired number of samples per class is not available without replacement, CARAT falls back to replacement and, if needed, to a random subset of the probe pool rather than aborting the round. This fail-soft design preserves the intended semantic diversity of the task sampler while ensuring that the server always obtains a well-defined certificate matrix.

Robust score calibration. For notation economy, the main text reuses C after standardization. In implementation terms, the raw certificate matrix is first transformed column-wise into

$$C^{\mathrm{std}}_{i,t} = \frac{c_{i,t} - \mathrm{median}_j(c_{j,t})}{1.4826 \cdot \mathrm{MAD}_j(c_{j,t}) + \varepsilon}.$$

This removes task-specific scale effects while preserving within-task ordering by Proposition B.4. Rank penalties are calibrated similarly. For each coordinate subsample S_s, let

$$\rho_i^{(s)} = \frac{1}{|S_s|} \sum_{j \in S_s} \left| \frac{\mathrm{rank}_{i,j}}{n-1} - \frac{1}{2} \right|.$$

CARAT then computes a standardized rank score

$$z_i^{(s)} = \frac{\rho_i^{(s)} - \mathrm{median}_\ell(\rho_\ell^{(s)})}{1.4826 \cdot \mathrm{MAD}_\ell(\rho_\ell^{(s)}) + \varepsilon},$$

and aggregates subsamples through

$$r_i = \max\left(0, \mathrm{median}_s\, z_i^{(s)}\right).$$

As a result, a client is penalized only when it is consistently extreme across repeated coordinate views, not because of a single noisy subsample.

Projected optimization details. Again writing C for the standardized certificate matrix, CARAT initializes the optimizer with $\alpha^{(0)} = \alpha^{\mathrm{prev}}$ after the first round, and with the uniform vector at initialization. If

$$z(\alpha) = C^\top \alpha, \qquad \pi_t(\alpha) = \frac{\exp(-\beta z_t(\alpha))}{\sum_{\ell=1}^{T} \exp(-\beta z_\ell(\alpha))},$$

then the ascent direction used in projected updates is

$$g(\alpha) = C\pi(\alpha) - \lambda_r r - 2\lambda_p(\alpha - \alpha^{\mathrm{prev}}).$$

With optimizer step size η, one iteration takes the form

$$\alpha^{(k+1)} = \Pi_{\mathcal{A}(\hat{\rho})}\left(\alpha^{(k)} + \eta g(\alpha^{(k)})\right).$$

The projection maintains feasibility throughout optimization, so every intermediate iterate already obeys the nonnegativity, unit-sum, and per-client cap constraints described in the main text.

Round invariants and failure handling. Two implementation choices are important for robustness. First, the optimizer never leaves the capped simplex, so every intermediate and final weight vector remains feasible. Second, probe-task construction is fail-soft rather than fail-stop: if strict class-balanced sampling is impossible, the code falls back to replacement or to a random subset, ensuring that the round can still proceed with a well-defined certificate matrix.

Outputs. Aggregated server update, updated temporal prior α^{prev}, and diagnostic logs including learned weights, task scores, softmin focus, rank penalties, evaluation radius, clipping threshold, and the identities of the hidden probe tasks used in the current round.

Practical diagnostics. The most useful round-level diagnostics for debugging and empirical interpretation are the learned weights, the weakest-task values emphasized by the softmin, the clip threshold τ_r, the evaluation radius q_r, and the final rank penalties r_i. In successful runs, these quantities help distinguish three qualitatively different cases: benign but diverse clients that retain moderate weight, clearly harmful clients that lose certificate support across multiple tasks, and structurally extreme clients whose influence is reduced primarily by the rank penalty.

B Theoretical Analysis

This appendix records the main structural properties used to interpret CARAT. These results are not presented as a complete robustness guarantee against arbitrary adaptive attackers. Instead, they clarify why the method decomposes naturally into a normalized evaluation geometry, a task-aware utility objective, and a constrained robust aggregation step.

Proposition B.1 (Feasibility and support of the capped simplex)**.** *Let*

$$\mathcal{A}(\hat{\rho}) = \left\{ \alpha \in \mathbb{R}^n : \alpha_i \geq 0,\ \sum_{i=1}^{n} \alpha_i = 1,\ \alpha_i \leq \frac{1}{(1-\hat{\rho})n} \right\},$$

with $\hat{\rho} < 1$. Then $\mathcal{A}(\hat{\rho})$ is non-empty, and every $\alpha \in \mathcal{A}(\hat{\rho})$ satisfies

$$|\operatorname{supp}(\alpha)| \geq (1-\hat{\rho})n.$$

Proof sketch. Let $u = 1/((1-\hat{\rho})n)$. Since $\hat{\rho} < 1$, we have $u \geq 1/n$, so the uniform weight vector is feasible and the set is non-empty. For the support bound, if only k coordinates of α are positive, then

$$1 = \sum_{i=1}^{n} \alpha_i \leq ku = \frac{k}{(1-\hat{\rho})n},$$

which implies $k \geq (1-\hat{\rho})n$. Therefore CARAT can suppress suspicious clients, but cannot collapse the aggregate onto an arbitrarily small subset of participants. □

Proposition B.2 (Softmin bounds and gradient form)**.** *Let $s \in \mathbb{R}^T$ and define*

$$\operatorname*{softmin}_{\beta}(s) = -\frac{1}{\beta} \log \left(\frac{1}{T} \sum_{t=1}^{T} e^{-\beta s_t} \right).$$

Then

$$\min_t s_t \leq \operatorname*{softmin}_{\beta}(s) \leq \min_t s_t + \frac{\log T}{\beta}.$$

Moreover,

$$\lim_{\beta \to \infty} \operatorname*{softmin}_{\beta}(s) = \min_t s_t, \qquad \lim_{\beta \to 0} \operatorname*{softmin}_{\beta}(s) = \frac{1}{T} \sum_{t=1}^{T} s_t.$$

Moreover, if $z(\alpha) = C^\top \alpha$ and

$$\pi_t(\alpha) = \frac{\exp(-\beta z_t(\alpha))}{\sum_{\ell=1}^{T} \exp(-\beta z_\ell(\alpha))},$$

then

$$\nabla_\alpha \operatorname*{softmin}_{\beta}(C^\top \alpha) = C\pi(\alpha).$$

Proof sketch. Let $m = \min_t s_t$. Since

$$e^{-\beta m}/T \leq \frac{1}{T} \sum_{t=1}^{T} e^{-\beta s_t} \leq e^{-\beta m},$$

applying $-(1/\beta)\log(\cdot)$ yields

$$m \le \operatorname*{softmin}_{\beta}(s) \le m + \frac{\log T}{\beta}.$$

The limit $\beta \to \infty$ follows immediately from the vanishing gap bound. For $\beta \to 0$, a first-order expansion of the exponential average gives

$$\frac{1}{T}\sum_{t=1}^{T} e^{-\beta s_t} = 1 - \beta \frac{1}{T}\sum_{t=1}^{T} s_t + o(\beta),$$

so the softmin converges to the arithmetic mean. Differentiating the softmin expression with respect to z_t yields the Gibbs weights $\pi_t(\alpha)$, and the chain rule then gives $C\pi(\alpha)$. Hence the ascent direction is a convex combination of certificate columns, with more weight placed on tasks whose current aggregate utility is smaller. □

Proposition B.3 (Directional invariance of certificate scoring)**.** *Suppose two nonzero clipped updates satisfy $\bar{\Delta}_i = a\bar{\Delta}_j$ for some scalar $a > 0$. Then their normalized versions in Eq.* (3) *coincide, i.e.,*

$$\tilde{\Delta}_i = \tilde{\Delta}_j.$$

Consequently, their hidden-task certificates are identical:

$$c_{i,t} = c_{j,t} \qquad \textit{for all } t \in \{1, \dots, T\}.$$

Proof sketch. Eq. (3) rescales every nonzero clipped update to the common radius q_r. Positive collinearity is therefore removed by normalization:

$$q_r \frac{\bar{\Delta}_i}{\|\bar{\Delta}_i\|_2} = q_r \frac{a\bar{\Delta}_j}{\|a\bar{\Delta}_j\|_2} = q_r \frac{\bar{\Delta}_j}{\|\bar{\Delta}_j\|_2}.$$

Substituting the resulting equality into Eq. (4) gives identical task scores. Thus, after clipping, the certificate stage depends on direction rather than residual norm magnitude. □

Proposition B.4 (Task-wise standardization preserves within-task ordering)**.** *Fix a task t and define a standardized certificate score*

$$\hat{c}_{i,t} = \frac{c_{i,t} - m_t}{s_t + \varepsilon},$$

where $m_t \in \mathbb{R}$ and $s_t > 0$ depend only on task t and not on client i. Then for any clients i, j,

$$c_{i,t} \ge c_{j,t} \quad \Longleftrightarrow \quad \hat{c}_{i,t} \ge \hat{c}_{j,t}.$$

Moreover,

$$\hat{c}_{i,t} - \hat{c}_{j,t} = \frac{c_{i,t} - c_{j,t}}{s_t + \varepsilon}.$$

Proof sketch. For a fixed task t, the map $x \mapsto (x - m_t)/(s_t + \varepsilon)$ is affine with strictly positive slope because $s_t + \varepsilon > 0$. Hence it preserves ordering. The difference identity follows by direct subtraction. Therefore column-wise robust standardization calibrates cross-task scale without changing the within-task ranking induced by the raw certificate scores. □

Proposition B.5 (Temporal prior and projected-iterate invariants)**.** *Define*

$$f(\alpha) = \operatorname*{softmin}_{\beta}(C^\top \alpha) - \lambda_r r^\top \alpha - \lambda_p \|\alpha - \alpha^{\text{prev}}\|_2^2.$$

If $\alpha, \alpha' \in \mathcal{A}(\hat{\rho})$ satisfy $C^\top \alpha = C^\top \alpha'$ and $r^\top \alpha = r^\top \alpha'$, then

$$f(\alpha) - f(\alpha') = -\lambda_p \left(\|\alpha - \alpha^{\text{prev}}\|_2^2 - \|\alpha' - \alpha^{\text{prev}}\|_2^2 \right).$$

In addition, under projected gradient ascent

$$\alpha^{(k+1)} = \Pi_{\mathcal{A}(\hat{\rho})}\left(\alpha^{(k)} + \eta \nabla f(\alpha^{(k)}) \right),$$

every iterate remains in $\mathcal{A}(\hat{\rho})$.

Proof sketch. The first statement follows by direct substitution: if the certificate and rank terms match, the only remaining difference in objective value is the quadratic prior term. CARAT therefore prefers, among utility-equivalent feasible weights, the solution closer to α^{prev}. The second statement is immediate from the projection operator. Hence the optimizer preserves three round invariants throughout: $\alpha_i \geq 0$, $\sum_i \alpha_i = 1$, and $\alpha_i \leq 1/((1-\hat{\rho})n)$. □

Proposition B.6 (Norm bound for the final aggregate)**.** *Suppose each clipped update satisfies* $\|\bar{\Delta}_i\|_2 \leq \tau_r$ *and* $\alpha \in \mathcal{A}(\hat{\rho})$*. Then the final aggregate*

$$\Delta_{\text{agg}} = \sum_{i=1}^{n} \alpha_i \bar{\Delta}_i$$

satisfies

$$\|\Delta_{\text{agg}}\|_2 \leq \tau_r.$$

Proof sketch. Because $\alpha_i \geq 0$ and $\sum_i \alpha_i = 1$, the aggregate is a convex combination of clipped updates. By the triangle inequality,

$$\|\Delta_{\text{agg}}\|_2 \leq \sum_{i=1}^{n} \alpha_i \|\bar{\Delta}_i\|_2 \leq \sum_{i=1}^{n} \alpha_i \tau_r = \tau_r.$$

Thus the final server step inherits the clipping radius as a hard norm upper bound. □

Proposition B.7 (Per-round complexity)**.** *Let n be the number of clients, T the number of hidden tasks, m the number of sampled coordinates for rank scoring, S the number of rank subsamples, and K the number of projected-gradient steps. Ignoring constant factors from batch size and model width, the extra defense-side work of CARAT scales as*

$$\textit{probe-model forward passes} + O(Snm + KnT).$$

Proof sketch. CARAT performs one baseline probe evaluation plus n candidate probe evaluations over the T hidden tasks, then computes S rank summaries over m coordinates, and finally executes K projected optimization steps over n weights and T task scores. In the current implementation, the probe-model forward passes dominate wall-clock time, while the rank and optimization stages are lower-order terms controlled by m, S, and K. □

Remark on assumptions. The theoretical object D^{ref} in the main text should be read as the probe pool *after* subsampling. The code, however, obtains that pool by sampling from the full server-visible training dataset. Any final formal discussion should keep this distinction explicit, because it materially strengthens the server-side assumption relative to a fully data-free defense.

C Additional Experimental Details

This section records setup details and benchmark scope that are omitted from the main text for brevity.

Detailed benchmark scope. The broader repository configuration includes three suites. The clean-reference suite uses `NoAttack` and compares {Mean, FLTrust, MultiKrum, CARAT}. The untargeted poisoning suite evaluates {ALIE, MinMax, FangAttack} against the same defense set. The backdoor suite evaluates {BadNets, Neurotoxin} against {Mean, FLTrust, FLAME, DeepSight, CARAT}. The committed configs cover both i.i.d. and non-i.i.d. settings: `clean_reference.yaml` and `main_untargeted.yaml` use $\alpha \in \{0.1, 0.5\}$, while `backdoor.yaml` uses i.i.d. plus $\alpha = 0.5$. The untargeted suite sweeps attacker fractions $\{0.2, 0.4\}$; the clean-reference and backdoor suites use 0 and 0.2, respectively. Each planned suite runs three repeats with effective seeds 42, 43, and 44. The current main text intentionally reports only the completed untargeted slice rather than this full benchmark matrix.

Default CARAT hyperparameters. Unless otherwise stated, CARAT uses probe pool size 512, 8 hidden tasks, 4 probe samples per class, probe batch size 128, probe-radius quantile 0.5, MAD clipping with $k = 2.5$, default attacker prior $\hat{\rho} = 0.2$, certificate temperature $\beta = 12.0$, rank weight 0.20, prior weight 0.05, 4096 sampled coordinates, 5 rank subsamples, 60 optimization steps, and optimizer learning rate 0.5.

Currently verified run. The strongest directly verified completed CARAT run in the repository uses FedAvg, CIFAR100, ResNet-18, an i.i.d. partition, 20 clients, 4 attackers, FangAttack, 300 rounds, learning rate 0.05, local epochs 4, and `evaluate=false`. The main text uses this run only as preliminary evidence, because the broader clean-reference, backdoor, non-i.i.d., and multi-seed CARAT suites are still being completed.

Next additions.

1. Complete the running `main_untargeted.yaml` sweep, prioritizing the CARAT rows and reporting multi-seed means and variances for clean test accuracy and loss.
2. Extend the verified runs from the current i.i.d. slice to the non-i.i.d. Dirichlet settings already present in `main_untargeted.yaml`.
3. Complete the committed `backdoor.yaml` suite so that the paper reports both clean utility and ASR.
4. Run the planned ablations, prioritizing the number of hidden tasks, the rank and prior weights, and the clipping controls.
5. Report multi-seed means and variances, plus defense runtime overhead, before making any strong comparative claim against other robust aggregators.